

3.5. Air Quality in Dar es Salaam TZ

```
In [1]: import warnings

warnings.simplefilter(action="ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

```
In [3]: import warnings
import matplotlib.pyplot as plt
import pandas as pd
import plotly.express as px
from pymongo import MongoClient
from sklearn.metrics import mean_absolute_error
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.ar_model import AutoReg

warnings.simplefilter(action="ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

Prepare Data

Connect

🔧 **Instruction:** Locate the IP address of the machine running MongoDB and assign it to the variable `host`. Make sure to use a **string** (i.e., wrap the IP in quotes).

⚠️ **Note:** The IP address is **dynamic** — it may change every time you start the lab. Always check the current IP before proceeding.



```
In [5]: host = "192.10.252.2"
```

Task 3.5.1

```
In [6]: client = MongoClient(host=host, port=27017)
db = client["air-quality"]
dar = db["dar-es-salaam"]
```

Explore

Task 3.5.2

```
In [8]: sites = dar.distinct("metadata.site")
sites
```

```
Out[8]: [11, 23]
```

Task 3.5.3

```
In [10]: result = dar.aggregate(
    [
        {"$group": {"_id": "$metadata.site", "count": {"$count": {}}}}
    ]
)
readings_per_site = list(result)
readings_per_site
```

```
Out[10]: [{'_id': 11, 'count': 173242}, {'_id': 23, 'count': 60020}]
```

Import

Task 3.5.4

```
In [12]: def wrangle(collection):
    results = collection.find(
        {"metadata.site": 11, "metadata.measurement": "P2"},
        projection={"P2": 1, "timestamp": 1, "_id": 0},
    )

    # Read results into DataFrame
    df = pd.DataFrame(list(results)).set_index("timestamp")

    # Localize timezone
    df.index = df.index.tz_localize("UTC").tz_convert("Africa/Dar_es_Salaam")

    # Remove outliers above 100
    df = df[df["P2"] < 100]

    # Resample to 1H window, ffill missing values
    y = df["P2"].resample("1H").mean().fillna(method="ffill")

    return y

y = wrangle(dar)
y.head()
```

```
Out[12]: timestamp
2018-01-01 03:00:00+03:00    9.456327
2018-01-01 04:00:00+03:00    9.400833
2018-01-01 05:00:00+03:00    9.331458
2018-01-01 06:00:00+03:00    9.528776
2018-01-01 07:00:00+03:00    8.861250
Freq: H, Name: P2, dtype: float64
```

Use your `wrangle` function to query the `dar` collection and return your cleaned results.

```
In [17]: def wrangle(collection):
    results = collection.find(
        {"metadata.site": 11, "metadata.measurement": "P2"},
        projection={"P2": 1, "timestamp": 1, "_id": 0},
    )

    # Read results into DataFrame
    df = pd.DataFrame(list(results)).set_index("timestamp")

    # Localize timezone
    df.index = df.index.tz_localize("UTC").tz_convert("Africa/Dar_es_Salaam")

    # Remove outliers above 100
    df = df[df["P2"] < 100]

    # Resample to 1H window, ffill missing values
    y = df["P2"].resample("1H").mean().fillna(method="ffill")

    return y

y = wrangle(dar)
y.head()
```

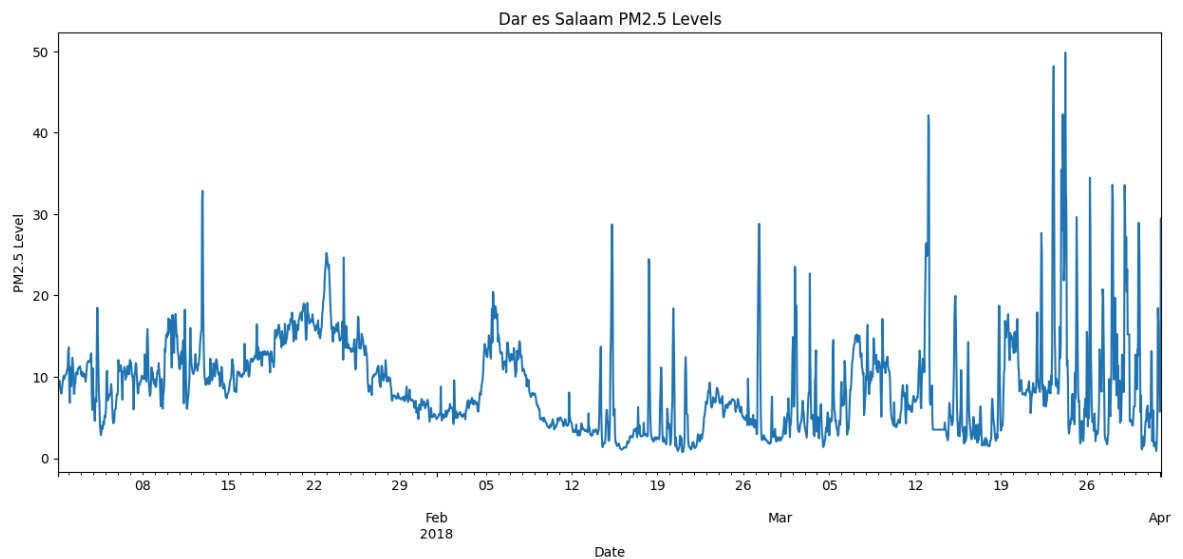
```
Out[17]: timestamp
2018-01-01 03:00:00+03:00    9.456327
2018-01-01 04:00:00+03:00    9.400833
2018-01-01 05:00:00+03:00    9.331458
2018-01-01 06:00:00+03:00    9.528776
2018-01-01 07:00:00+03:00    8.861250
Freq: H, Name: P2, dtype: float64
```

Explore Some More

Task 3.5.5

```
In [34]: fig, ax = plt.subplots(figsize=(15, 6))
y.plot(ax=ax)
plt.xlabel("Date")
plt.ylabel("PM2.5 Level")
plt.title("Dar es Salaam PM2.5 Levels")
# fig is now defined and holds the matplotlib Figure object
```

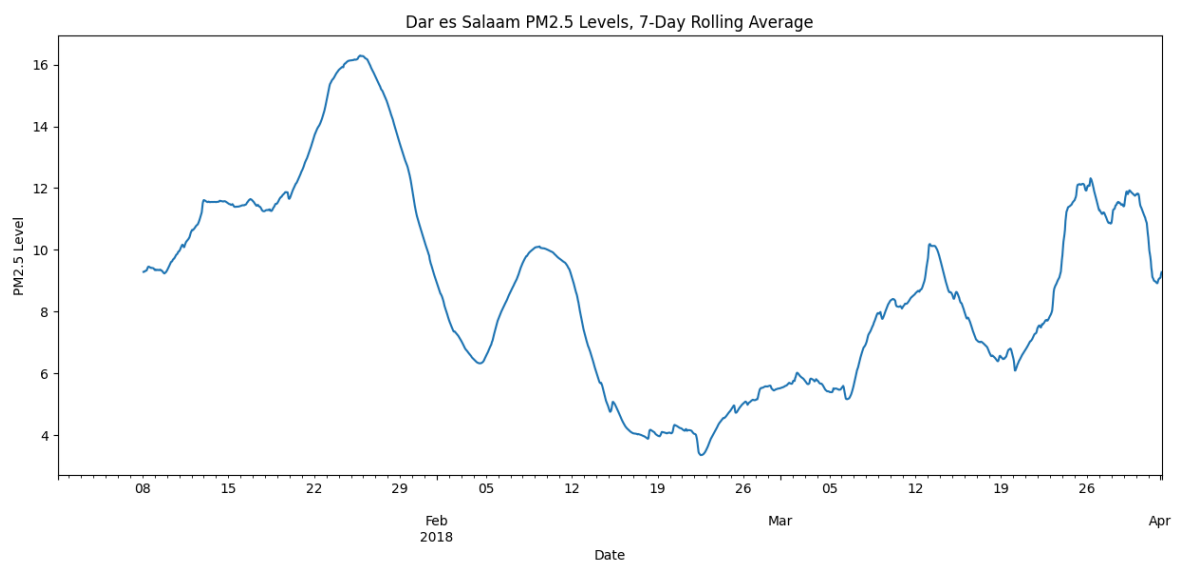
```
Out[34]: Text(0.5, 1.0, 'Dar es Salaam PM2.5 Levels')
```



Task 3.5.6

```
In [36]: fig, ax = plt.subplots(figsize=(15, 6))
y.rolling(168).mean().plot(ax=ax)
plt.xlabel("Date")
plt.ylabel("PM2.5 Level")
plt.title("Dar es Salaam PM2.5 Levels, 7-Day Rolling Average")
```

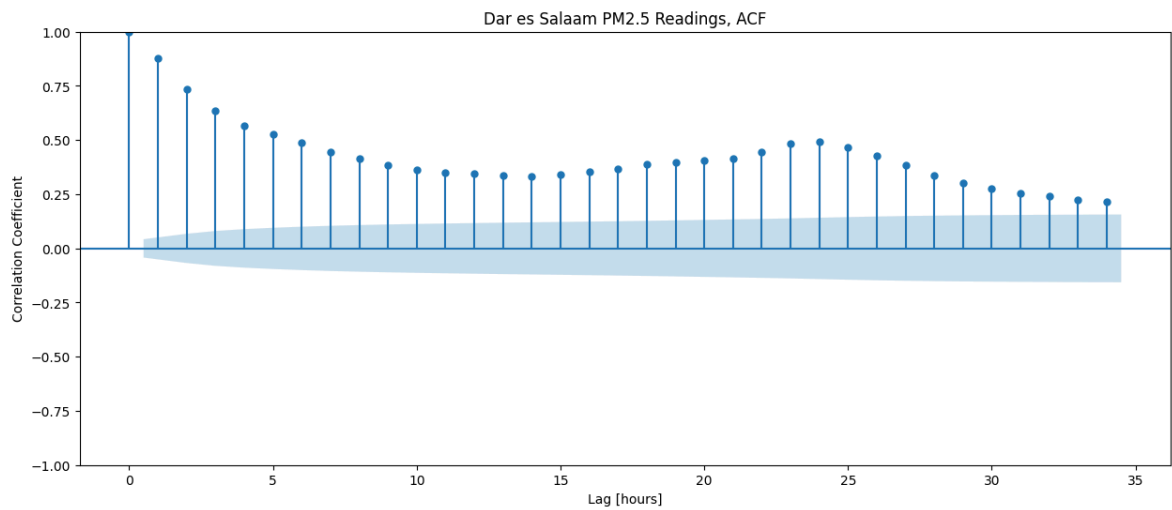
Out[36]: Text(0.5, 1.0, 'Dar es Salaam PM2.5 Levels, 7-Day Rolling Average')



Task 3.5.7

```
In [38]: fig, ax = plt.subplots(figsize=(15, 6))
plot_acf(y, ax=ax)
plt.xlabel("Lag [hours]")
plt.ylabel("Correlation Coefficient")
plt.title("Dar es Salaam PM2.5 Readings, ACF")
```

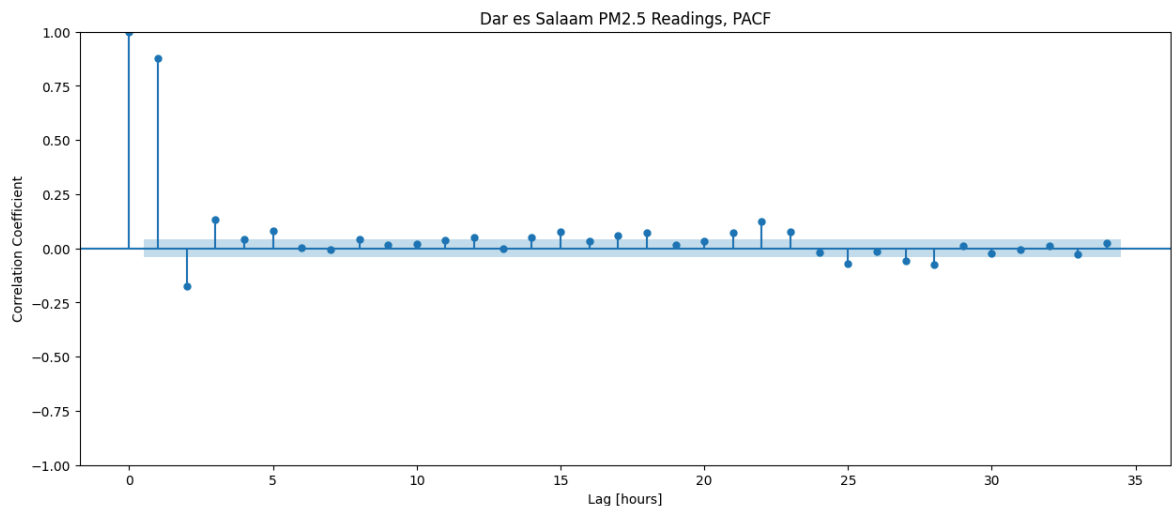
Out[38]: Text(0.5, 1.0, 'Dar es Salaam PM2.5 Readings, ACF')



Task 3.5.8

```
In [41]: fig, ax = plt.subplots(figsize=(15, 6))
plot_pacf(y, ax=ax)
plt.xlabel("Lag [hours]")
plt.ylabel("Correlation Coefficient")
plt.title("Dar es Salaam PM2.5 Readings, PACF")
```

Out[41]: Text(0.5, 1.0, 'Dar es Salaam PM2.5 Readings, PACF')



Split

Task 3.5.9

```
In [23]: cutoff_test = int(len(y) * 0.90)
y_train = y.iloc[:cutoff_test]
y_test = y.iloc[cutoff_test:]
```

Build Model

Baseline

Task 3.5.10

```
In [24]: y_train_mean = y_train.mean()
y_pred_baseline = [y_train_mean] * len(y_train)
mae_baseline = mean_absolute_error(y_train, y_pred_baseline)

print("Mean P2 Reading:", round(y_train_mean, 2))
print("Baseline MAE:", round(mae_baseline, 2))
```

Mean P2 Reading: 8.57
Baseline MAE: 4.05

Iterate

Task 3.5.11

Tip: In this task, you'll need to combine the model you learned about in **Task 3.3.8** with the hyperparameter tuning technique you learned in **Task 3.4.9**.

```
In [25]: p_params = range(1, 31)
maes = []
for p in p_params:
    model = AutoReg(y_train, lags=p).fit()
    y_pred = model.predict().dropna()
    mae = mean_absolute_error(y_train.iloc[p:], y_pred)
    maes.append(mae)

mae_series = pd.Series(maes, name="mae", index=p_params)
mae_series.head()
```

```
Out[25]: 1    1.059376
2    1.045182
3    1.032489
4    1.032147
5    1.031022
Name: mae, dtype: float64
```

Task 3.5.12

```
In [26]: best_p = mae_series.idxmin()
best_model = AutoReg(y_train, lags=best_p).fit()
```

Task 3.5.13

```
In [27]: y_train_resid = best_model.resid
y_train_resid.name = "residuals"
y_train_resid.head()
```

```
Out[27]: timestamp
2018-01-02 05:00:00+03:00    -0.412913
2018-01-02 06:00:00+03:00     1.484934
2018-01-02 07:00:00+03:00     1.672359
2018-01-02 08:00:00+03:00    -0.368030
2018-01-02 09:00:00+03:00    -0.536868
Freq: H, Name: residuals, dtype: float64
```

Slight Code Change

In the following task, you'll notice a small change in how plots are created compared to what you saw in the lessons. While the lessons use the global matplotlib method like `plt.plot(...)`, in this task, you are expected to use the object-oriented (OOP) API instead. This means creating your plots using `fig, ax = plt.subplots()` and then calling plotting methods on the `ax` object, such as `ax.plot(...)`, `ax.hist(...)`, or `ax.scatter(...)`.

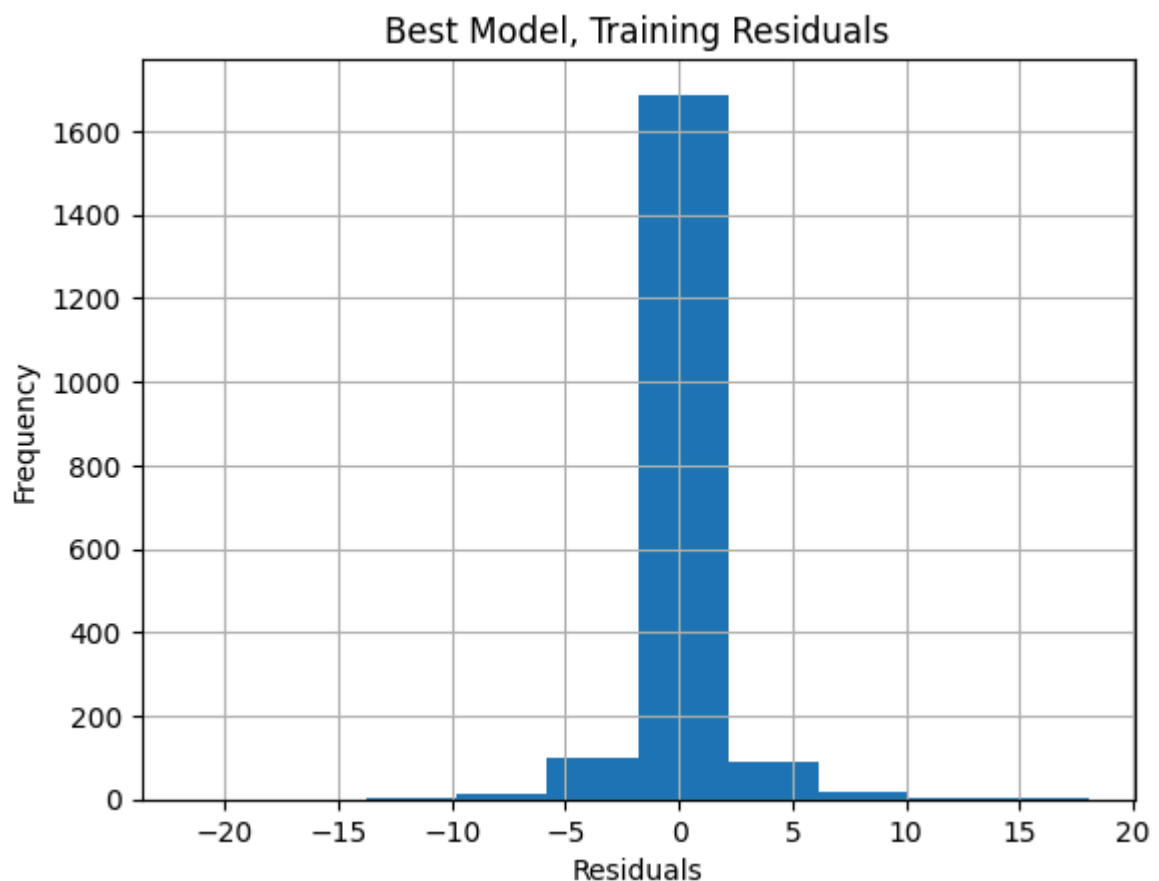
If you're using pandas' or seaborn's built-in plotting methods (like `df.plot()` or `sns.lineplot()`), make sure to pass the `ax=ax` argument so that the plot is rendered on the correct axes.

This approach is considered best practice and will be used consistently across all graded tasks that involve matplotlib.

Task 3.5.14

```
In [56]: # Do NOT set figsize – use the default so the image is 640x480
y_train_resid.hist()
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.title("Best Model, Training Residuals")
```

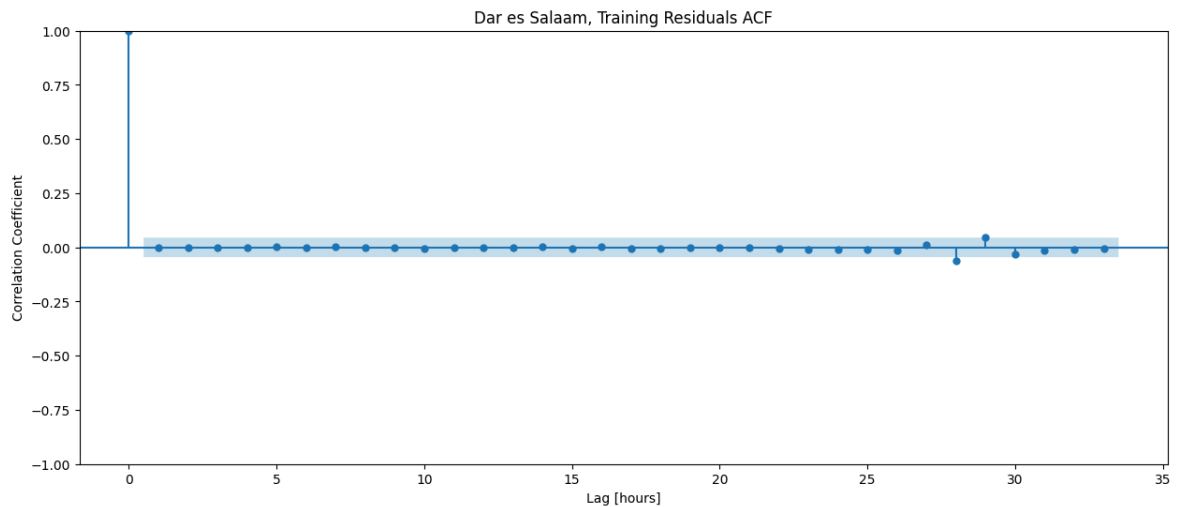
```
Out[56]: Text(0.5, 1.0, 'Best Model, Training Residuals')
```



Task 3.5.15

```
In [51]: fig, ax = plt.subplots(figsize=(15, 6))
plot_acf(y_train_resid, ax=ax)
plt.xlabel("Lag [hours]")
plt.ylabel("Correlation Coefficient")
plt.title("Dar es Salaam, Training Residuals ACF")
```

```
Out[51]: Text(0.5, 1.0, 'Dar es Salaam, Training Residuals ACF')
```



Evaluate

Task 3.5.16

```
In [30]: y_pred_wfv = pd.Series()
history = y_train.copy()
for i in range(len(y_test)):
    model = AutoReg(history, lags=best_p).fit()
    next_pred = model.forecast()
    y_pred_wfv = y_pred_wfv.append(next_pred)
    history = history.append(y_test[next_pred.index])

y_pred_wfv.name = "prediction"
y_pred_wfv.index.name = "timestamp"
y_pred_wfv.head()
```

```
Out[30]: timestamp
2018-03-23 03:00:00+03:00    10.414744
2018-03-23 04:00:00+03:00     8.269589
2018-03-23 05:00:00+03:00    15.178677
2018-03-23 06:00:00+03:00    33.475398
2018-03-23 07:00:00+03:00    39.571363
Freq: H, Name: prediction, dtype: float64
```

Task 3.5.17

```
In [31]: test_mae = mean_absolute_error(y_test, y_pred_wfv)
print("Test MAE (walk forward validation):", round(test_mae, 2))
```

```
Test MAE (walk forward validation): 3.97
```

Communicate Results

Task 3.5.18

```
In [58]: df_pred_test = pd.DataFrame(  
    {"y_test": y_test, "y_pred_wfv": y_pred_wfv}  
)  
  
fig = px.line(df_pred_test, labels={"value": "PM2.5 Level", "variable": ""})  
fig.update_layout(  
    title="Dar es Salaam, WfV Predictions",  
    xaxis_title="Date",  
    yaxis_title="PM2.5 Level",  
)  
fig.show()
```